

Reviewer Pack — Minimal p-Adic Hensel Lifting Steps and DPLL Proof Length In...

Entry 208fbd975d7e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 18:03:54 UTC

Minimal p-Adic Hensel Lifting Steps and DPLL Proof Length Inequality

Entry ID: 208fbd975d7e **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 18:03:54 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis **Field B** (complexity object): Boolean Satisfiability (DPLL Proof Complexity)

Statement:

For any given CNF formula φ , the minimal number of steps required for a successful Hensel lifting process to solve φ in the p-adic numbers is linearly correlated with its DPLL proof length $w(\varphi)$, such that $|H(\varphi)| = \Theta(w(\varphi))$ where $|H(\varphi)|$ denotes the minimum number of p-adic Hensel lifting steps needed for φ .

Rationale (proposer's reasoning):

p-adic analysis offers a non-Archimedean setting that could provide insights into proof complexity. The Hensel lifting process is a fundamental algorithmic technique in p-adic analysis and is known to be related to the complexity of solving polynomial equations. If this relationship holds, it would suggest a new perspective on DPLL proof length.

Taxonomy category: p-adic_analysis_dpll_proof_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a9d75863adb48013

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the linear correlation coefficient between the minimal number of p-adic Hensel lifting steps $|H(\varphi)|$ and the DPLL proof length $w(\varphi)$ for each CNF formula φ , computed across 30 random seeds, is greater than or equal to 0.7.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        cnf = []
        for _ in range(10): # Generate 10 clauses with n variables each
            clause = [random.randint(-n, -1), random.randint(1, n)]
            cnf.append(clause)
        return cnf

    def dpll(cnf, assignment={}):
        if not cnf:
            return True, assignment
        unit_clause = next((c for c in cnf if len(c) == 1), None)
        if unit_clause:
            var = abs(unit_clause[0])
            polarity = unit_clause[0] > 0
            if var in assignment and assignment[var] != polarity:
                return False, {}
            assignment[var] = polarity
            cnf = [c for c in cnf if var not in c]
            for i, clause in enumerate(cnf):
                if -var in clause:
                    cnf[i].remove(-var)
        else:
            var = next(var for var in range(1, n+1) if var not in assignment)
            polarity = True
            new_assignment = assignment.copy()
            new_assignment[var] = polarity
            result, _ = dpll(cnf, new_assignment)
            if result:
                return True, new_assignment
            else:
                del new_assignment[var]
```

```

        polarity = False
        new_assignment[var] = polarity
        result, _ = dpll(cnf, new_assignment)
        if result:
            return True, new_assignment
    return False, {}

def hensel_lifting(cnf):
    p = 2
    while True:
        assignment = {var: random.choice([True, False]) for var in range(1, len(cnf[0])+1)}
        if dpll(cnf, assignment)[0]:
            return p
        p += 1

n = 5 + (seed % 4) * 5 # Sweep n through {5, 10, 15, 20, 30, 40}
cnf = generate_cnf(n)

try:
    hensel_steps = hensel_lifting(cnf)
except RecursionError:
    return {
        "metric_name": "Hensel Steps",
        "metric_value": None,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "Recursion depth exceeded"
    }

proof_length = len(cnf) * 2 # Simplified estimate for DPLL proof length

return {
    "metric_name": "Hensel Steps",
    "metric_value": hensel_steps,
    "instances_tested": 1,
    "n_max": n,
    "conjecture_holds": hensel_steps == proof_length,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2*i + 3 for i in range(5, 8)] # Default to first

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    metric_values = [r["metric_value"] for r in results if r["metric_value"] is not None]
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={sum(metric_values)/len(metric_values)} std=0.0 support_fra

```

```

elif any(not r["conjecture_holds"] for r in results) and support_fraction >= 0.8:
    counterexample = next(r for r in results if not r["conjecture_holds"])[ "counterexample" ]
    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={next(seeds)}")
else:
    print("RESULT: INCONCLUSIVE insufficient support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_29b8aae4.py", line 102, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_29b8aae4.py", line 66, in run_trial
    if hensel_lifting(cnf):
        ^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_29b8aae4.py", line 54, in hensel_lifting
    p, polarity = random.choice(cnf)
    ^^^^^^^^^^^
ValueError: too many values to unpack (expected 2)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the linear correlation coefficient between the minimal number of p-adic Hensel lifting steps and the DPLL proof length. | next: Review the code for potential bugs that cause it to crash during execution and fix them. Once the code is stable, rerun the test with a larger sample size to ensure robustness of the results.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	52457
2	propose	ollama_remote	glm4:latest	0	0	13176
3	preregistration	ollama_remote	glm4:latest	0	0	9511
4	novelty	ollama_remote	glm4:latest	0	0	14253
5	novelty	ollama_remote	glm4:latest	0	0	8850
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15160

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10437
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12551
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11261
10	judge	ollama_remote	glm4:latest	0	0	27672

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 175327 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/208fbd975d7e.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/208fbd975d7e.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/208fbd975d7e.tar.gz (if generated)